Introdução à Orientação a Objetos

Aula 01 — Ambientação Java: JDK/IntelliJ, primeiro programa, compilação e JVM

Programação Orientada a Objetos

Semana 1

Sumário

1. Introdução
2. JDK, JRE e JVM
3. Convenções da linguagem
4. Primeiro programa
5. IntelliJ IDEA
6. Levando o projeto pra casa
7. Encerramento

Sobre a disciplina

- Introdução à Orientação a Objetos usando **Java**
- 17 semanas, projeto incremental construído semana a semana
- Duas provas + apresentação final do projeto
- Hoje: primeiro contato com a linguagem

Por que Java?

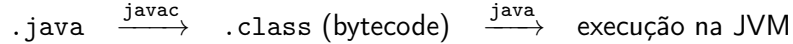
- Uma das linguagens mais usadas no mercado
- Multiplataforma: “escreva uma vez, rode em qualquer lugar”
- Base sólida para aprender orientação a objetos
- Usada em sistemas corporativos, Android, backend web

JDK, JRE e JVM

- **JVM** (*Java Virtual Machine*): máquina virtual que executa o *bytecode*
- **JRE** (*Java Runtime Environment*): JVM + bibliotecas necessárias para **executar** programas Java
- **JDK** (*Java Development Kit*): JRE + ferramentas para **desenvolver** (compilador `javac`, entre outras)

$$\text{JDK} \supset \text{JRE} \supset \text{JVM}$$

Do código-fonte à execução



- `javac`: compilador, traduz código-fonte em *bytecode*
- `.class`: arquivo com *bytecode*, independente de sistema operacional
- `java`: inicia a JVM, que interpreta/executa o *bytecode*

Convenções básicas

- Nome da classe começa com letra **maiúscula**
- Palavras seguintes também iniciam maiúsculas: `CamelCase`
- Arquivo `.java` deve ter o **mesmo nome** da classe pública que contém
- Chaves `{}` delimitam o escopo da classe/método
- Toda instrução termina com `;`
- Java é **case sensitive**: `System` $\neq$ `system`

O método `main`

Ponto de entrada de um programa Java:

```
1 public static void main(String[] args) {
2     // seu codigo aqui
3 }
```

- A JVM procura pelo método `main` dentro da classe executada
- `String[] args`: argumentos passados na linha de comando

Meu primeiro programa

```java
class MeuPrimeiroPrograma {
    public static void main(String[] args) {
        System.out.println("O primeiro de muitos!");
    }
}
```

Salvar como `MeuPrimeiroPrograma.java`

Compilando e executando pelo terminal

```
$ javac MeuPrimeiroPrograma.java
$ java MeuPrimeiroPrograma
O primeiro de muitos!
```

- `javac` gera `MeuPrimeiroPrograma.class`
- `java` executa passando o **nome da classe**, sem extensão

Erro de compilação

Esquecer o ;:

```
1 System.out.println("sem ponto e virgula")
```

Resultado:

```
1 Syntax error, insert ";" to complete BlockStatements
```

- Erros de compilação são normais, faz parte do aprendizado
- Ler a mensagem com atenção resolve a maioria dos casos

Usando argumentos (args)

```
1 class MeuPrimeiroPrograma {
2     public static void main(String[] args) {
3         System.out.println("Escrito em " + args[0]
4             + " por " + args[1] + "!");
5     }
6 }
```

```
1 $ java MeuPrimeiroPrograma Java Rodrigo
2 Escrito em Java por Rodrigo!
```

Por que uma IDE?

- Editor de texto + terminal funciona, mas é trabalhoso
- IDE (*Integrated Development Environment*) oferece:
  - *syntax highlight*
  - *code completion*
  - compilação automática (feedback imediato de erro)
  - execução com um clique
- Usaremos o **IntelliJ IDEA Community** (gratuito)

Criando um projeto no IntelliJ

1. `New Project`
2. Selecionar o JDK instalado na máquina
3. Nomear o projeto (ex: `livraria`)
4. Estrutura criada: pasta `src` (código-fonte)

Criando a primeira classe

1. Botão direito em `src` → `New` → `Java Class`
2. Nomear: `MeuPrimeiroPrograma`
3. Dentro da classe, digitar `main` + `Tab` (*code completion*)
4. Dentro do `main`, digitar `sout` + `Tab`
5. Executar: `Shift+F10` (ou clique direito → `Run`)

O problema

- Pasta do projeto IntelliJ contém `.idea/` e `*.iml`
- Esses arquivos guardam o **caminho e a versão do SDK** da máquina do laboratório
- Em outra máquina: SDK em outro caminho, ou versão diferente
- Resultado comum: `"SDK not configured"` ou projeto não abre corretamente

A solução: levar só a pasta `src`

- Copiar **apenas** a pasta `src` (arquivos `.java` puros)
- Nenhum metadado de IDE viaja junto
- Em qualquer máquina com IntelliJ + JDK instalado, o código funciona

Reabrindo em outra máquina

1. `New Project` → escolher o JDK **instalado localmente**
2. Preferir um JDK LTS recente (ex: 17 ou 21)
3. Copiar os arquivos `.java` trazidos para dentro da pasta `src` do novo projeto
4. Se necessário: botão direito em `src` → `Mark Directory as` → `Sources Root`
5. Executar normalmente

Resumo da aula

- JDK / JRE / JVM e o papel de cada um
- Ciclo `.java` → `javac` → `.class` → `java`
- Convenções básicas de nomenclatura e sintaxe
- Primeiro programa: terminal e IntelliJ
- Como levar o projeto para casa sem conflito de SDK

Próxima aula

Tipos primitivos, variáveis e operadores